Object-Oriented Design Pattern Laboratory Examination
Project 1: Agricultural Automation System (AAS)

Suggested duration: 3 hours
Total marks: 100%
Written examination: Phases 1–4 — 50% marks
Java implementation: Phase 5 — 50% marks

Project scenario

A farm currently relies on manual monitoring of soil moisture, irrigation, fertilizer application, and crop health. This causes delayed decisions, unnecessary water consumption, and difficulties responding to changing weather conditions.

Develop an **Agricultural Automation System (AAS)** that monitors farm conditions, supports irrigation decisions, and sends notifications to farmers. The system should support different sensor types, alternative irrigation strategies, and integration with an external weather service.

Consider the following actors: **Farmer, Farm Administrator, IoT Sensor, Weather Service, Drone System, and AI Analytics System**. Clearly state which features your examination solution implements and which remain outside its scope.

Instructions

- Complete **Phases 1–4 on paper** and **Phase 5 in Java**.
- Keep class names and responsibilities consistent across your written design and implementation.
- Use simulated sensor readings and weather data. Actual hardware, internet access, a database, and AI model training are not required.
- Implement a console application demonstrating the core irrigation workflow.
- State reasonable assumptions wherever requirements are unspecified.

Phase 1: Requirement Analysis — 10% marks

Question 1

Analyze the Agricultural Automation System and complete the following tasks:

a. Describe the system background and identify **three problems** with the existing manual farming process. **[2 marks]**

b. State **one primary objective** and **four specific objectives** of the proposed system. **[2 marks]**

c. Identify **four actors** and explain how each interacts with the system. Define the system boundary so that external devices and services are clearly distinguished from internal software components. **[2 marks]**

d. Specify **four functional requirements** and **two non-functional requirements**. At least two functional requirements must concern the irrigation workflow you will implement. **[4 marks]**

Phase 2: Object Identification — 10% marks

Question 2

Identify and classify suitable objects for the AAS.

a. Identify **five entity classes** and provide at least **two attributes** for each class. **[4 marks]**

b. Identify **three control classes** and explain their responsibilities. **[3 marks]**

c. Identify **three boundary classes** and explain the external interaction each supports. **[3 % marks]**

Present your answer using the following format:

Class name	Classification	Attributes or responsibilities
Your identified class	Entity / Control / Boundary	Relevant details

You may consider classes such as Crop, Field, SoilSensor, WeatherData, IrrigationPump, IrrigationController, FarmerMobileApp, and WeatherAPIInterface. Justify your classification rather than simply copying the examples.

Phase 3: CRC Card Development — 10 % marks

Question 3

Prepare **six CRC cards** for classes involved in monitoring soil conditions, obtaining weather information, making irrigation decisions, and notifying the farmer.

Each card must include:

- **Class name**
- **At least two meaningful responsibilities**
- **Collaborating classes**

Your cards must collectively include at least **one entity, one control, and one boundary class**. Ensure that the collaborators support the responsibilities assigned to each class.

Phase 4: UML Modeling — 20 % marks

Question 4

a. UML class diagram — 12 % marks

Draw a UML class diagram for the core irrigation subsystem containing **at least eight classes or interfaces**.

Include:

- Class and interface names.
- Important attributes and methods.
- Visibility indicators.
- Associations and relevant multiplicities.
- Interface realization or inheritance where appropriate.
- Classes supporting your selected design patterns.

Show aggregation or composition **only where the ownership and lifecycle relationship justifies it**.

b. UML sequence diagram — 8 % marks

Draw a sequence diagram for the following operation:

A farmer requests an irrigation check. The system reads soil moisture, obtains weather information through an adapter, applies the selected irrigation strategy, activates or leaves the pump off, and sends the farmer a notification.

Include:

- The farmer and relevant software objects.
- Messages in execution order.
- An alt fragment showing **irrigation required** and **irrigation not required**.
- A notification following the decision.

Phase 5: Java Implementation — 50 % marks

Question 5

Implement a console-based irrigation subsystem using **all five design patterns** below.

Pattern	Required implementation	% marks
Singleton	Implement FarmDataManager with a private constructor and a controlled access method. Store simulated farm information and demonstrate that two access calls return the same instance.	6
Factory	Implement SensorFactory to create at least two sensor types, such as SoilMoistureSensor and TemperatureSensor, through a common Sensor interface. Handle unsupported sensor types. A Simple Factory is acceptable; identify the variant used.	8
Observer	Implement an alert publisher and at least two subscribers, such as FarmerNotifier and FarmDashboard. Demonstrate subscription, notification, and unsubscription.	8
Strategy	Implement an IrrigationStrategy interface with NormalIrrigationStrategy and WaterSavingStrategy. Allow the controller to switch strategies at runtime.	8

Pattern	Required implementation	% marks
Adapter	Implement WeatherService and a WeatherAPIAdapter that wraps a simulated external weather class with an incompatible method or data format.	8
Integration and testing	Integrate the patterns into a runnable irrigation workflow and demonstrate the required tests below.	12
Total		**50**

Required irrigation rules

Use these **simplified examination rules**, rather than treating them as agricultural recommendations:

- **Normal strategy:** Irrigate when soil moisture is **below 40%** and rain is **not expected**.
- **Water-saving strategy:** Irrigate when soil moisture is **below 25%** and rain is **not expected**.
- Otherwise, leave the pump off.
- Reject soil-moisture readings outside **0–100%**.

The controller must use weather information obtained through the adapter and publish the irrigation decision to its registered observers.

Required test demonstrations

Test	Input or action	Expected behavior
1	Moisture 30%, no rain, normal strategy	Pump on
2	Moisture 30%, no rain, water-saving strategy	Pump off
3	Moisture 20%, rain expected, either strategy	Pump off
4	Moisture 40%, no rain, normal strategy	Pump off
5	Moisture 25%, no rain, water-saving strategy	Pump off
6	Moisture −5% or 105%	Invalid reading rejected; pump not activated
7	Publish an alert with two observers registered	Both observers receive the alert

Test	Input or action	Expected behavior
8	Unsubscribe one observer and publish another alert	Only the remaining observer receives it
9	Request FarmDataManager twice	Both references identify the same instance
10	Create both supported sensor types and request an unsupported type	Correct sensors returned; unsupported request handled clearly

Submission requirements

Submit the handwritten answers for Phases 1–4, Java source files for Phase 5, brief compilation and execution instructions, and a test-results table showing expected and actual outcomes.

Dhaka Urban Management System Lab Examination

Project 2 Questions and Model Solutions

Complete all five phases. Write Phases 1–4 on paper and implement Phase 5 in Java. This document provides an examination paper without marks, followed by a model solution. Alternative designs are acceptable when their analysis, diagrams, and implementation are consistent.

Part A Examination Questions

Project scenario

Design a Dhaka Urban Management System that integrates urban services, citizen communication, IoT monitoring, and decision support. The broader platform aims to manage traffic, improve waste collection, handle complaints, support emergency response, monitor environmental conditions, and generate urban reports.

For the examination, implement a complaint-to-service workflow: a citizen submits a complaint, the system creates the appropriate urban service, selects a response route using external traffic data, updates the complaint status, and notifies subscribers.

Instructions

- Complete Phases 1–4 on paper and Phase 5 as a Java console application.
- Simulate complaints, routes, traffic information, and notifications. A live database, internet service, physical sensors, and a graphical interface are not required.
- Use all five specified patterns. Clearly distinguish the broader platform from the implemented subsystem.
- Use consistent class names and state reasonable assumptions.

Phase 1 Requirement Analysis

Question 1 Analyze the proposed system

- Describe the system background and identify three problems the proposed system addresses.
- State one primary objective and four specific objectives.
- Identify at least four actors and their responsibilities. Define the boundary between the platform and external devices or services.
- Specify four functional requirements and two non-functional requirements. At least two functional requirements must concern complaint handling and service dispatch.

Phase 2 Object Identification

Question 2 Identify entity control and boundary objects

- Identify at least five entity classes, giving two or more attributes for each.
- Identify three control classes and explain their responsibilities.
- Identify three boundary classes and explain their external interactions.
- Present the classifications in tables and explain any difference between an external actor and its internal software representation.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

Prepare six CRC cards for the complaint-handling and service-dispatch workflow. Include class name, at least two responsibilities, and collaborating classes. Across the six cards, include at least one entity, one control, and one boundary class.

Phase 4 UML Modeling

Question 4 Model the system structure and behavior

a. Draw a UML class diagram with at least eight classes or interfaces. Include important attributes, methods, visibility indicators, relevant associations and multiplicities, and interface realizations. Show the participants in the selected patterns. Use aggregation or composition only when ownership and lifecycle semantics justify it.

b. Draw a sequence diagram for dispatching a response to an OPEN complaint. Show complaint retrieval, service creation, external traffic retrieval through an adapter, route selection, the status change to IN_PROGRESS, and observer notification. Use an alt fragment for a route available versus no route available. When no route exists, the complaint must remain OPEN.

Phase 5 Java Implementation

Question 5 Implement and demonstrate five design patterns

Pattern	Required implementation
Singleton	UrbanDatabase provides controlled access to one shared in-memory complaint repository. Demonstrate instance identity.
Factory	UrbanServiceFactory creates WasteService, TrafficService, or EmergencyService through UrbanService. Reject unknown types. A Simple Factory is acceptable; name the variant.
Observer	Publish complaint status changes to CitizenNotifier and GovernmentDashboard. Demonstrate subscription, removal, and citizen-specific notification filtering.
Strategy	RoutePlanner switches between ShortestRouteStrategy and FastestRouteStrategy at runtime.
Adapter	TrafficAPIAdapter translates a simulated external traffic API into the TrafficServiceAPI interface, converting metres to kilometres and seconds to minutes.

Examination rules

Use status transitions OPEN → IN_PROGRESS → RESOLVED. Submission creates OPEN; dispatch changes OPEN to IN_PROGRESS only after a route is obtained; resolution is allowed only from IN_PROGRESS. Reject duplicate complaint IDs, missing IDs, unsupported service types, and invalid transitions.

Choose among three candidate routes to the same destination. Route A: 5 km, 30 minutes; Route B: 8 km, 20 minutes; Route C: 6.5 km, 25 minutes. Shortest chooses minimum distance; fastest chooses minimum estimated time. Break ties by route ID in ascending lexical order. Reject an empty candidate list.

Required Demonstrations and Submission

Test	Input or action	Expected behavior
1	Request repository twice	Same instance
2	Create WASTE, TRAFFIC, EMERGENCY	Correct concrete service types
3	Request UNKNOWN service	Reject request
4	Submit valid complaint	Stored with OPEN status
5	Repeat ID or use blank ID	Reject without a new complaint
6	Convert route A provider values	5000 m → 5 km; 1800 s → 30 min
7	Switch shortest to fastest	A first, then B
8	Dispatch valid OPEN complaint	Selected route; IN_PROGRESS; observers notified
9	Resolve an OPEN complaint	Reject; remains OPEN
10	Unsubscribe dashboard then resolve	RESOLVED; citizen notified, dashboard not notified
11	Dispatch RESOLVED or missing ID	Reject; no valid transition
12	No candidate routes during dispatch	Reject; complaint remains OPEN

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and test results showing expected and actual behavior. The candidate-route values above are fictional examination data, not observations of Dhaka traffic.

Smart University Campus Management System Lab Examination

Project 3 Questions and Model Solutions

Complete five phases: write Phases 1–4 on paper and implement Phase 5 in Java. This document contains questions without marks followed by an instructor model solution. Other technically justified designs are acceptable.

Part A Examination Questions

Project scenario

An integrated university platform manages student information, registration, classrooms, schedules, security, notices, and digital services. For the examination, implement account creation, course registration, student evaluation, targeted result notifications, and a simulated external library lookup. Classroom scheduling, RFID attendance, transport, and security monitoring remain extensions.

General instructions

- Use a Java console application with simulated data. A graphical interface, live services, physical devices, and a persistent database are not required.
- Apply Singleton, Factory, Observer, Strategy, and Adapter. Keep class names consistent across analysis, CRC cards, UML, and code.
- State assumptions and distinguish the full platform from the implemented subsystem.

Phase 1 Requirement Analysis

Question 1 Analyze the requirements

- Describe the background and three problems to be addressed.
- State one primary objective and four specific objectives.
- Identify at least four actors, explain their interactions, and define the system boundary.
- Specify four functional and two non-functional requirements linked to the implemented use cases.

Phase 2 Object Identification

Question 2 Classify the objects

Identify at least five entity classes with two or more attributes each, three control classes with responsibilities, and three boundary classes with external interactions. Present three tables and distinguish external actors from internal software representations.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

For six important classes, list the class name, at least two responsibilities, and collaborators. Include at least one entity, one control, and one boundary class across the cards.

Phase 4 UML Modeling

Question 4 Model structure and interactions

a. Draw a UML class diagram containing at least eight classes or interfaces. Show attributes, operations, visibility, appropriate multiplicities, associations, and interface realizations. Identify pattern participants. Use aggregation or composition only when ownership and lifecycle semantics justify it.

b. Model a faculty request to evaluate validated marks and publish a personal result. Show mark validation, strategy delegation, publication, and recipient filtering. Use alternatives for valid versus invalid marks; invalid marks must stop before result publication.

Phase 5 Java Implementation

Question 5 Implement all five design patterns

Pattern	Required behavior
Singleton	UniversityConfig supplies one shared configuration instance with a six-credit registration limit.
Factory	AccountFactory creates Student, Faculty, or Administrator through UserAccount; reject unsupported roles. A Simple Factory is acceptable.
Observer	NoticePublisher supports subscription and removal; broadcast notices to portals and filter personal results by recipient ID.
Strategy	EvaluationService switches between TheoryEvaluation and LabEvaluation at runtime.
Adapter	LibraryAdapter implements LibraryService, translating text copy counts from ExternalLibrarySystem into availability.

Examination rules

- Maximum registration load is six credits. Reject duplicate registration, full courses, and a registration exceeding the limit. A load of exactly six is allowed.
- Theory score = 0.60 × exam + 0.40 × coursework. Lab score = 0.30 × exam + 0.20 × coursework + 0.50 × project. Each input must be finite and between 0 and 100 inclusive.
- With exam 80, coursework 70, project 90, the theory score is 76 and the lab score is 83. These are examination policies, not an institution’s official grading rules.
- A public notice has no recipient ID and reaches all subscribed portals. A personal result reaches only the matching student portal; the example faculty portal displays public notices only.
- The simulated library returns BOOK1 → "2", BOOK2 → "0", and unknown IDs → "UNKNOWN". The adapter returns true, false, or an explicit error respectively.

Required Demonstrations and Submission

Area	Required demonstration
Configuration and factory	Same configuration instance; all three account types created; unknown role and blank ID rejected.
Registration	Register two three-credit courses; reject a duplicate, full course, and a third three-credit course. Failed registrations leave credit totals unchanged.
Evaluation	Theory gives 76; after switching policy, lab gives 83. Reject −1, 101, and NaN marks.
Observer	Broadcast reaches all portals; result for S1 does not display in S2 or the faculty portal. Removed S2 portal receives no later broadcast.
Library adapter	BOOK1 available; BOOK2 unavailable; unknown book rejected.

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and a test table comparing expected with actual results. Use the supplied fictional values consistently.

Part B Model Solutions

The model solution is one coherent design. It implements the examination scope, while some domain classes remain extensions. The complete Java listing uses nested classes to allow compilation from one source file.

Phase 1 Background and Objectives

Assume student information and digital services are handled in separate tools. Three problems are inconsistent registration checks, repeated manual score calculation, and poorly targeted notifications. The examination solution centralizes registration decisions and separates evaluation and integration logic behind interfaces.

Primary objective: Develop an object-oriented campus subsystem that validates registrations, computes scores using interchangeable policies, sends appropriate notifications, and accesses an external library through a stable interface.

Specific objectives

- Create supported account roles through one creation interface.
- Enforce registration capacity, duplication, and credit constraints.
- Calculate and publish scores using a selected evaluation policy.
- Integrate library availability and distinguish public notices from personal results.

Phase 1 Actors and Requirements

Actor	Interaction
Student	Registers for courses, receives notices and personal results, and checks book availability.
Faculty Member	Supplies assessment marks and chooses the appropriate evaluation policy.
Administrator	Creates account roles and maintains campus configuration and course information.
Librarian	Maintains the external library catalogue used through an adapter.

System boundary: the platform contains domain records, controllers, portals, and adapters. External people, third-party systems, and physical devices lie outside it. A portal or adapter is an internal boundary component, not the external actor itself.

Security Officer and IoT System are broader-platform actors for access monitoring and attendance. The model does not implement their hardware interactions.

Functional requirements

- FR1: Create Student, Faculty, and Administrator accounts through UserAccount and reject unsupported roles.
- FR2: Register a student only when the course is not already selected, a seat is available, and the resulting load is at most six credits.
- FR3: Validate assessment marks, compute scores with the selected policy, and notify the matching student.
- FR4: Broadcast notices, remove subscribers, and translate external library counts into availability.

Non functional requirements

- Maintainability: evaluation algorithms and library-provider integration must be replaceable through interfaces.
- Reliability: rejected registration must not change enrollment or credit totals; invalid marks must not produce a result notification.

Assumptions

One canonical Course object exists for each course code within a registration manager. Student IDs identify the same student across operations. The configuration is immutable, the program is single-threaded, and all data are in memory. Portal setup and faculty actions are simulated by main(). Recipient filtering demonstrates routing; it does not implement authentication or access control.

Phase 2 Object Identification Solution

Entity objects

Entity	Attributes	Implementation scope
Student	id, role	Implemented account and domain representation; role returned by role().
Course	code, credits, capacity, students	Implemented course information and enrolled student IDs.
Marks	exam, coursework, project	Implemented assessment value object.
LibraryBook	isbn, title, availableCopies	Extension; provider exposes copy counts in the example.
Notice	noticeId, message, recipientId	Extension; code passes recipient and message directly.

Control objects

Class	Responsibility
CourseRegistrationManager	Validate and record course registration and credit totals.
EvaluationService	Delegate calculation to the active policy and publish a result.
NoticePublisher	Coordinate registration, removal, and delivery of notification events.

Boundary objects

Class	Interaction
StudentPortal	Display public notices and results matching the student ID.
FacultyPortal	Display public campus notices in this teaching example.
LibraryAdapter	Bridge the incompatible external library query format.

Phase 3 CRC Card Solution

CourseRegistrationManager

Responsibilities: Check duplicate registration, capacity, and credit limit; update enrollment and credit totals.
Collaborators: Student, Course, UniversityConfig

Student

Responsibilities: Retain account identity; report the student role; identify the owner of registrations and results.
Collaborators: UserAccount, AccountFactory, CourseRegistrationManager, EvaluationService

Course

Responsibilities: Retain code, credits, and capacity; retain enrolled student IDs.
Collaborators: CourseRegistrationManager, Student

EvaluationService

Responsibilities: Replace evaluation policy; calculate scores; publish the score for its student.
Collaborators: EvaluationStrategy, Marks, Student, NoticePublisher

NoticePublisher

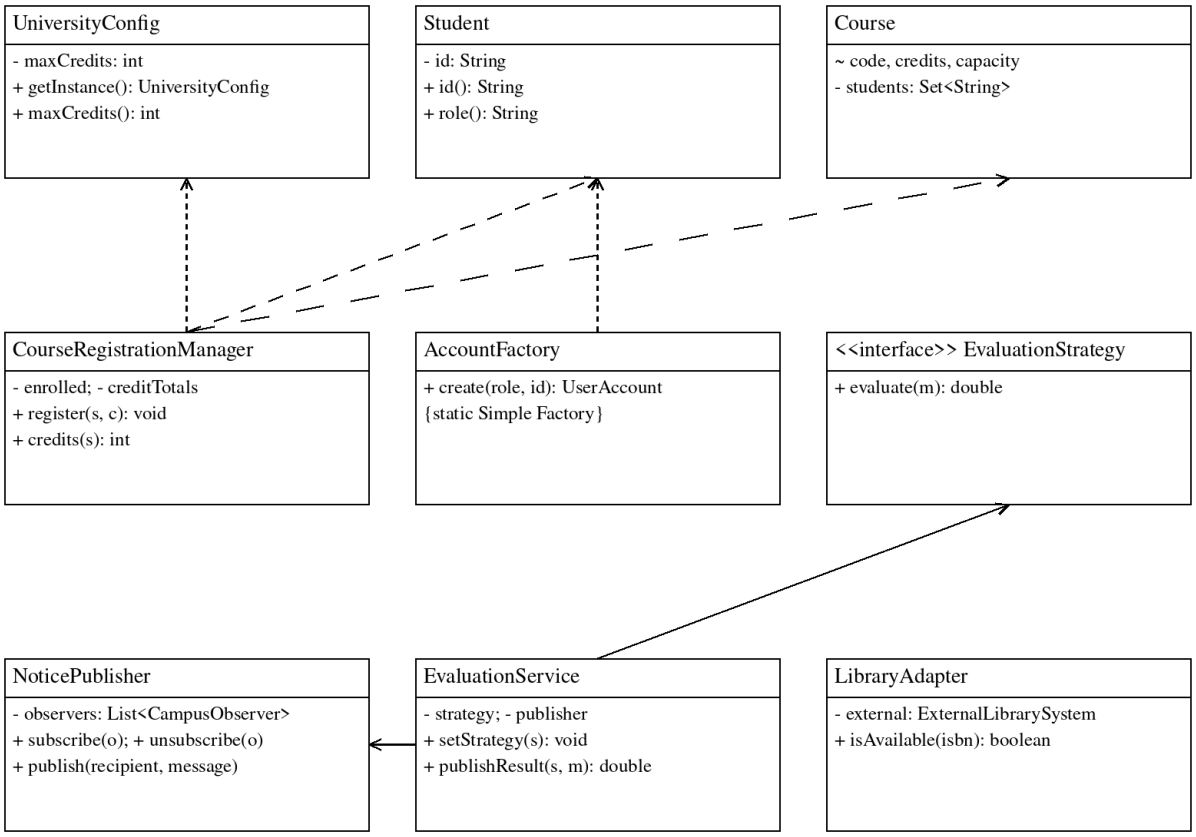
Responsibilities: Maintain subscribers; remove subscribers; deliver recipient-tagged messages.
Collaborators: CampusObserver, StudentPortal, FacultyPortal, EvaluationService

LibraryAdapter

Responsibilities: Query the provider; parse and validate copy counts; return availability or an error.
Collaborators: LibraryService, ExternalLibrarySystem

Phase 4 UML Class Diagram Solution

The diagram shows nine core classes or interfaces. Solid navigable lines indicate retained references and dashed arrows indicate method dependencies. The table completes concrete realizations and multiplicities. Package-visible fields use ~. Constructors and some helper details are omitted.

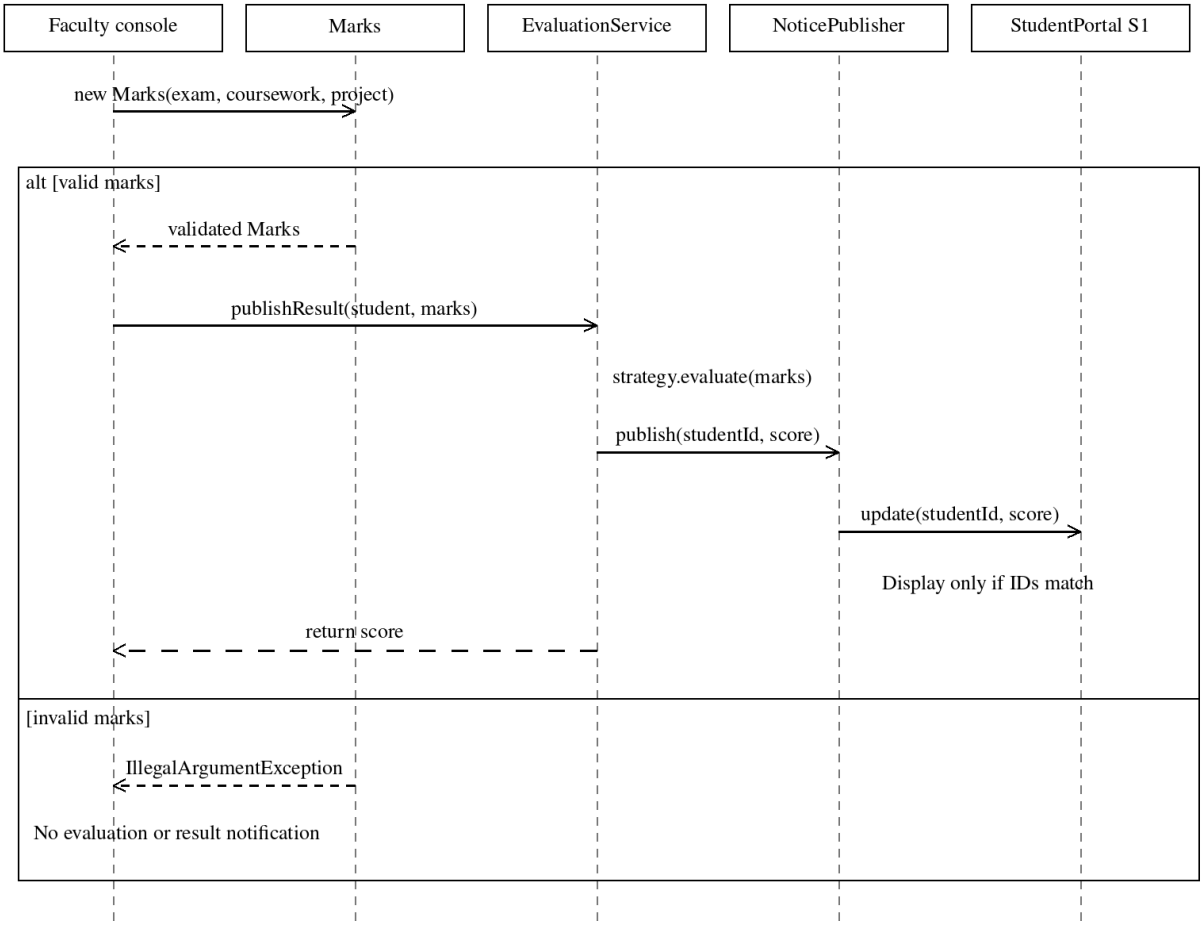


Additional interface realizations and associations are specified in the table below.

Relationship	Details
Factory participants	Student, Faculty, and Administrator realize UserAccount. AccountFactory depends on their constructors.
Registration relationships	One manager stores registrations for 0..* students; one Course records 0..* student IDs bounded by capacity. Each registration connects a student ID and a course code.
Observer participants	StudentPortal and FacultyPortal realize CampusObserver. One NoticePublisher stores 0..* observers; one EvaluationService holds one publisher.
Strategy participants	TheoryEvaluation and LabEvaluation realize EvaluationStrategy. Each EvaluationService holds one active strategy and depends on Marks and Student.
Adapter participants	LibraryAdapter realizes LibraryService and holds one ExternalLibrarySystem. Availability queries are independent of evaluation.

Phase 4 UML Sequence Diagram Solution

Model a faculty request to evaluate validated marks and publish a personal result. Show mark validation, strategy delegation, publication, and recipient filtering. Use alternatives for valid versus invalid marks; invalid marks must stop before result publication.



The faculty console constructs Marks before calling publishResult(). The Marks constructor rejects invalid inputs. EvaluationService delegates to its selected EvaluationStrategy; the diagram compresses this delegation into the calculation note. NoticePublisher calls update(recipient, message) on every registered observer. S1 displays its own result; S2 and the faculty portal filter this personal event. Filtering is an output rule rather than an authorization mechanism.

Associations and dependencies are sufficient for this model; no composition ownership is assumed. Keep the detailed message names aligned with the Java operations and the stated diagram simplifications.

Phase 5 Java Solution Listing 1

Save the complete listing as SUCMSExam.java. Compile with javac SUCMSExam.java and run java SUCMSExam. The code was compiled and executed with OpenJDK 17.0.20; all included checks passed. No third-party libraries are required.

The Singleton applies within this application and class-loader scope. The factory is a Simple Factory. The program is a sequential in-memory teaching simulation. Copy all listing sections in order into one source file.

```
import java.util.*;
public class SUCMSExam {
  static void requireText(String s) {
    if (s == null || s.trim().isEmpty())
      throw new IllegalArgumentException("Blank value");
  }
  static final class UniversityConfig {
    private static final UniversityConfig INSTANCE = new UniversityConfig();
    private final int maxCredits = 6;
    private UniversityConfig() { }
    public static UniversityConfig getInstance() { return INSTANCE; }
    public int maxCredits() { return maxCredits; }
  }
  interface UserAccount { String id(); String role(); }
  static final class Student implements UserAccount {
    private final String id;
    Student(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "STUDENT"; }
  }
  static final class Faculty implements UserAccount {
    private final String id;
    Faculty(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "FACULTY"; }
  }
  static final class Administrator implements UserAccount {
    private final String id;
```

Phase 5 Java Solution Listing 2

```
  Administrator(String id) { requireText(id); this.id = id; }
  public String id() { return id; }
  public String role() { return "ADMIN"; }
 }
 static final class AccountFactory {
  public static UserAccount create(String role, String id) {
   if ("STUDENT".equals(role)) return new Student(id);
   if ("FACULTY".equals(role)) return new Faculty(id);
   if ("ADMIN".equals(role)) return new Administrator(id);
   throw new IllegalArgumentException("Unsupported role");
  }
 }
 static final class Course {
  final String code;
  final int credits, capacity;
  private final Set<String> students = new HashSet<>();
  Course(String code, int credits, int capacity) {
   requireText(code);
   if (credits <= 0 || capacity <= 0) throw new IllegalArgumentException();
   this.code = code; this.credits = credits; this.capacity = capacity;
  }
 }
 static final class CourseRegistrationManager {
  private final Map<String, Set<String>> enrolled = new HashMap<>();
  private final Map<String, Integer> creditTotals = new HashMap<>();
  public void register(Student s, Course c) {
   Set<String> codes = enrolled.getOrDefault(s.id(), Collections.emptySet());
   int current = creditTotals.getOrDefault(s.id(), 0);
   if (codes.contains(c.code)) throw new IllegalStateException("Duplicate course");
   if (c.students.size() >= c.capacity) throw new IllegalStateException("Full course");
   if (current + c.credits > UniversityConfig.getInstance().maxCredits())
    throw new IllegalStateException("Credit limit");
   enrolled.computeIfAbsent(s.id(), key -> new HashSet<>()).add(c.code);
   c.students.add(s.id()); creditTotals.put(s.id(), current + c.credits);
  }
  public int credits(Student s) { return creditTotals.getOrDefault(s.id(), 0); }
 }
 interface CampusObserver { void update(String recipient, String message); }
 static final class StudentPortal implements CampusObserver {
  private final String studentId;
  int received;
  StudentPortal(String id) { studentId = id; }
  public void update(String recipient, String message) {
```

Phase 5 Java Solution Listing 3

```
    if (recipient != null && !recipient.equals(studentId)) return;
    received++; System.out.println("Student " + studentId + ": " + message);
  }
 }
 static final class FacultyPortal implements CampusObserver {
  int received;
  public void update(String recipient, String message) {
    if (recipient != null) return; // Only public notices in this example.
    received++; System.out.println("Faculty: " + message);
  }
 }
 static final class NoticePublisher {
  private final List<CampusObserver> observers = new ArrayList<>();
  public void subscribe(CampusObserver o) {
    if (!observers.contains(o)) observers.add(o);
  }
  public void unsubscribe(CampusObserver o) { observers.remove(o); }
  public void publish(String recipient, String message) {
    for (CampusObserver o : new ArrayList<>(observers)) o.update(recipient, message);
  }
 }
 static final class Marks {
  final double exam, coursework, project;
  Marks(double exam, double coursework, double project) {
    for (double x : new double[] {exam, coursework, project})
      if (!Double.isFinite(x) || x < 0 || x > 100)
        throw new IllegalArgumentException("Marks must be 0 to 100");
    this.exam = exam; this.coursework = coursework; this.project = project;
  }
 }
 interface EvaluationStrategy { double evaluate(Marks m); }
 static final class TheoryEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) { return .6 * m.exam + .4 * m.coursework; }
 }
 static final class LabEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) {
    return .3 * m.exam + .2 * m.coursework + .5 * m.project;
  }
 }
 static final class EvaluationService {
  private EvaluationStrategy strategy;
  private final NoticePublisher publisher;
  EvaluationService(EvaluationStrategy s, NoticePublisher p) {
```

Phase 5 Java Solution Listing 4

```
    strategy = s; publisher = p;
  }
  public void setStrategy(EvaluationStrategy s) { strategy = s; }
  public double publishResult(Student student, Marks marks) {
    double score = strategy.evaluate(marks);
    publisher.publish(student.id(), "Score=" + score); return score;
  }
}
interface LibraryService { boolean isAvailable(String isbn); }
static final class ExternalLibrarySystem {
  public String copiesAsText(String isbn) {
    if ("BOOK1".equals(isbn)) return "2";
    if ("BOOK2".equals(isbn)) return "0";
    return "UNKNOWN";
  }
}
static final class LibraryAdapter implements LibraryService {
  private final ExternalLibrarySystem external;
  LibraryAdapter(ExternalLibrarySystem external) { this.external = external; }
  public boolean isAvailable(String isbn) {
    requireText(isbn);
    String text = external.copiesAsText(isbn);
    if ("UNKNOWN".equals(text)) throw new IllegalArgumentException("Unknown book");
    int copies = Integer.parseInt(text);
    if (copies < 0) throw new IllegalArgumentException("Invalid copies");
    return copies > 0;
  }
}
static void expect(boolean ok, String label) {
  if (!ok) throw new AssertionError(label);
  System.out.println("PASS: " + label);
}
static void rejects(Runnable action, String label) {
  try { action.run(); }
  catch (IllegalArgumentException | IllegalStateException e) {
    System.out.println("PASS: " + label); return;
  }
  throw new AssertionError(label);
}
public static void main(String[] args) {
  expect(UniversityConfig.getInstance() == UniversityConfig.getInstance(), "singleton");
  Student student = (Student) AccountFactory.create("STUDENT", "S1");
  Student second = (Student) AccountFactory.create("STUDENT", "S2");
```

Phase 5 Java Solution Listing 5

```
    expect(AccountFactory.create("FACULTY", "F1") instanceof Faculty, "faculty factory");
    expect(AccountFactory.create("ADMIN", "A1") instanceof Administrator, "admin factory");
    rejects(() -> AccountFactory.create("UNKNOWN", "X"), "unsupported account");
    rejects(() -> AccountFactory.create("STUDENT", ""), "blank account ID");
    CourseRegistrationManager manager = new CourseRegistrationManager();
    Course c1 = new Course("C1", 3, 1), c2 = new Course("C2", 3, 5);
    Course c3 = new Course("C3", 3, 5);
    manager.register(student, c1);
    rejects(() -> manager.register(student, c1), "duplicate registration");
    rejects(() -> manager.register(second, c1), "capacity limit");
    manager.register(student, c2);
    expect(manager.credits(student) == 6, "six credit boundary accepted");
    rejects(() -> manager.register(student, c3), "credit limit exceeded");
    expect(manager.credits(student) == 6 && manager.credits(second) == 0,
        "failed registration leaves credits unchanged");
    NoticePublisher publisher = new NoticePublisher();
    StudentPortal p1 = new StudentPortal("S1"), p2 = new StudentPortal("S2");
    FacultyPortal faculty = new FacultyPortal();
    publisher.subscribe(p1); publisher.subscribe(p2); publisher.subscribe(faculty);
    publisher.publish(null, "Library hours updated");
    expect(p1.received == 1 && p2.received == 1 && faculty.received == 1,
        "broadcast notice");
    EvaluationService evaluation = new EvaluationService(new TheoryEvaluation(), publisher);
    Marks m = new Marks(80, 70, 90);
    expect(Math.abs(evaluation.publishResult(student, m) - 76) < 1e-9, "theory score 76");
    expect(p1.received == 2 && p2.received == 1 && faculty.received == 1,
        "result recipient filtering");
    evaluation.setStrategy(new LabEvaluation());
    expect(Math.abs(evaluation.publishResult(student, m) - 83) < 1e-9, "lab score 83");
    publisher.unsubscribe(p2); publisher.publish(null, "Campus notice");
    expect(p2.received == 1 && p1.received == 4 && faculty.received == 2,
        "observer removal");
    rejects(() -> new Marks(-1, 70, 90), "negative marks rejected");
    rejects(() -> new Marks(101, 70, 90), "marks above 100 rejected");
    rejects(() -> new Marks(Double.NaN, 70, 90), "NaN marks rejected");
    LibraryService library = new LibraryAdapter(new ExternalLibrarySystem());
    expect(library.isAvailable("BOOK1"), "available book");
    expect(!library.isAvailable("BOOK2"), "unavailable book");
    rejects(() -> library.isAvailable("MISSING"), "unknown book");
    System.out.println("All checks completed.");
  }
}
```

Testing Results and Design Interpretation

The main method executes checks and throws AssertionError on a mismatch. Its rejects() helper checks expected application exceptions. The following areas passed in the recorded Java run.

Test area	Observed behavior
Configuration and factory	Same configuration instance; all three account types created; unknown role and blank ID rejected.
Registration	Register two three-credit courses; reject a duplicate, full course, and a third three-credit course. Failed registrations leave credit totals unchanged.
Evaluation	Theory gives 76; after switching policy, lab gives 83. Reject −1, 101, and NaN marks.
Observer	Broadcast reaches all portals; result for S1 does not display in S2 or the faculty portal. Removed S2 portal receives no later broadcast.
Library adapter	BOOK1 available; BOOK2 unavailable; unknown book rejected.

Recorded output excerpts

PASS: singleton
PASS: six credit boundary accepted
PASS: failed registration leaves credits unchanged
PASS: theory score 76
PASS: result recipient filtering
PASS: lab score 83
PASS: observer removal
PASS: available book
PASS: unknown book
All checks completed.

These excerpts omit additional passing checks and interleaved notification messages. All checks completed. appears only after the earlier checks succeed.

Pattern rationale

The configuration Singleton centralizes an immutable policy. The Simple Factory isolates account construction. Evaluation strategies isolate scoring rules, observers separate publication from portal delivery, and the adapter isolates the library’s data representation.

Limitations

The example omits authentication, durable records, prerequisite checking, timetable conflicts, real library loans, transport, RFID hardware, and security-camera processing. The formulas and six-credit cap are fictional examination rules.

Object-Oriented Design Pattern Lab Manual

Phase 1: Requirement Analysis, 2

Phase 2: Object Identification, entity, control, boundary, 2

Phase 3: CRC Card Development, 2

Phase 4: UML Modeling, 2

Phase 5: Design Pattern Implementation: singleton patter, factory pattern, 7
Observer Pattern, Strategy Pattern,. Adapter Pattern

Object-Oriented Design Pattern Laboratory

Lab Title

Object-Oriented Analysis, Design, and Design Pattern Implementation for Real-World Software Systems

---

Lab Objective

The objective of this laboratory is to provide practical experience in analyzing real-world software problems, identifying objects, designing object interactions, modeling systems using UML, and implementing commonly used object-oriented design patterns.

After completing this laboratory, students will be able to:

- Analyze software requirements from real-world scenarios.
- Identify entity, control, and boundary objects.
- Develop CRC (Class Responsibility Collaborator) cards.
- Design UML class and sequence diagrams.
- Apply object-oriented design principles.
- Implement important design patterns using Java.

Laboratory Workflow

Students will complete the project through the following five phases:

Phase 1: Requirement Analysis

Objective

Understand the problem domain and identify the requirements of the proposed system.

Students need to analyze:

1. System Background

Describe:

- Why the system is required
- Current problems/challenges

- Expected software solution

Example:

Smart Agriculture Automation System

Traditional farming depends heavily on manual monitoring. Farmers face challenges in irrigation management, pest detection, weather uncertainty, and resource optimization. The proposed system integrates IoT sensors, AI analytics, and automated control mechanisms for intelligent farming.

---

2. System Objectives

Students should define:

- Primary objective
- Specific objectives

Example:

Primary Objective:

Develop an intelligent agriculture management platform for automated monitoring and decision support.

Specific Objectives:

- Monitor soil conditions
- Automate irrigation
- Detect crop diseases
- Provide weather-based recommendations

---

3. System Actors

Identify external users or systems interacting with the software.

Example:

Actor	Responsibility
Farmer	Monitor crops and receive alerts
Administrator	Manage system
Sensor	Provide environmental data
Weather Service	Provide weather information

Phase 2: Object Identification

Objective

Identify important objects from the problem domain.

Students classify objects into three categories:

2.1 Entity Objects

Definition

Entity objects represent real-world objects or information that the system needs to store and manage.

They usually represent the problem domain data.

Example:

Smart Agriculture System

Entity Object	Attributes
Crop	cropID, type, healthStatus
SoilSensor	sensorID, moisture, pH
IrrigationPump	pumpID, capacity, status
WeatherData	temperature, humidity

2.2 Control Objects

Definition

Control objects manage system operations, decision-making, and communication between objects.

They represent the system logic.

Example:

Control Object	Responsibility
IrrigationController	Decide irrigation timing
PestDetectionManager	Analyze pest information
HarvestPlanner	Determine harvesting time

Example:

IrrigationController

Reads SoilSensor data

↓

Checks moisture level

↓

Activates IrrigationPump

2.3 Boundary Objects

Definition

Boundary objects provide interaction between the system and external users/devices/services.

Example:

Boundary Object	Purpose
FarmerMobileApp	Farmer interaction
WeatherAPIInterface	External weather data
SensorGateway	IoT communication
AlertService	Notification delivery

Phase 3: CRC Card Development

Objective

Identify class responsibilities and relationships before UML design.

Students must prepare CRC cards for important classes.

CRC means:

- **Class**
- **Responsibilities**
- **Collaborators**

CRC Card Format

Class Name

Responsibilities

- Responsibility 1

- Responsibility 2

Collaborators

Example

CRC Card: IrrigationController

Class	IrrigationController
Responsibilities	- Monitor soil moisture - Decide irrigation requirement - Control irrigation pump
Collaborators	SoilSensor, IrrigationPump, WeatherData

Students should prepare minimum:

6 CRC Cards

for each assigned project.

Phase 4: UML Modeling

Objective

Convert object analysis into visual software design.

Students must prepare:

4.1 UML Class Diagram

The class diagram should contain:

- Class names
- Attributes
- Methods
- Relationships

Required relationships:

Association

Example:

A Farmer manages one or more Crops.

Aggregation

Example:

A Farm has many Sensors; the Sensors can exist independently of the Farm.

Composition

Example:

A House is composed of Rooms; the Rooms cannot exist without the House.

Generalization

Example:

Car and Truck are specialized types of Vehicle.

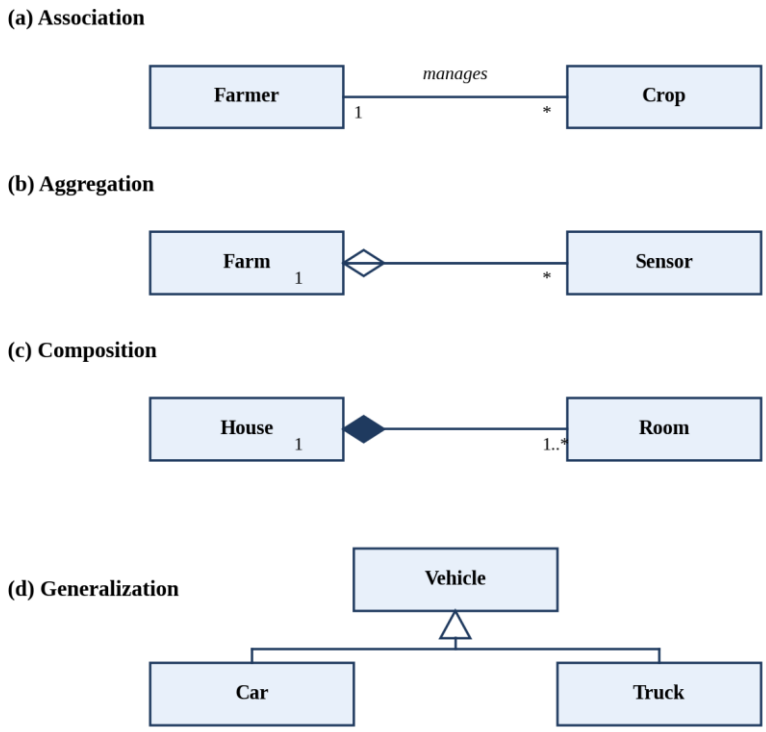


Figure 1: UML relationship notations: association, aggregation, composition, and generalization

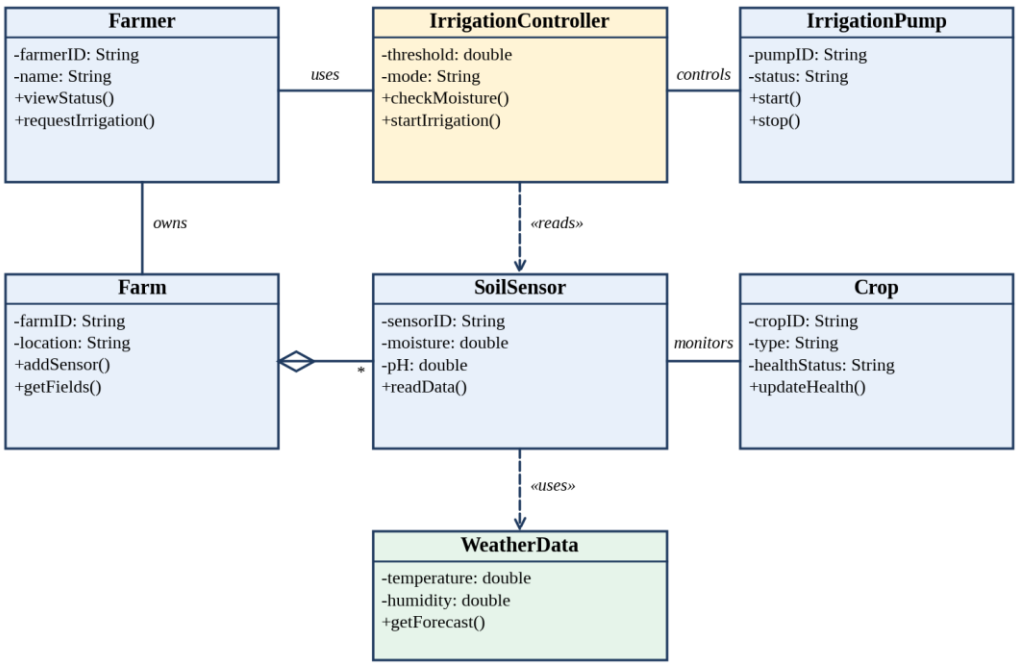


Figure 2: Sample UML class diagram for the Smart Agriculture Automation System

---

4.2 UML Sequence Diagram

Students should select one important system operation.

Example:

Smart Irrigation Workflow

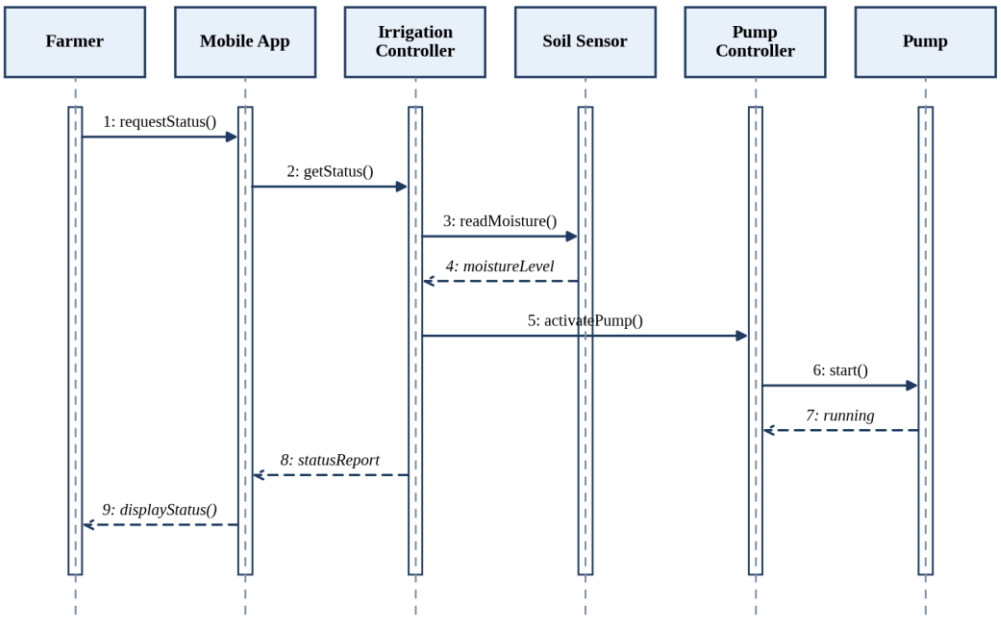


Figure 3: Sample UML sequence diagram: Smart Irrigation Workflow

The sequence diagram should show:

- Objects involved
- Message exchange
- Operation flow

Phase 5: Design Pattern Implementation

Students must implement the following five design patterns.

5.1 Singleton Pattern

Purpose

Ensure only one instance of a class exists throughout the system.

Possible Application

- Database Manager
- Configuration Manager
- System Logger

Example:

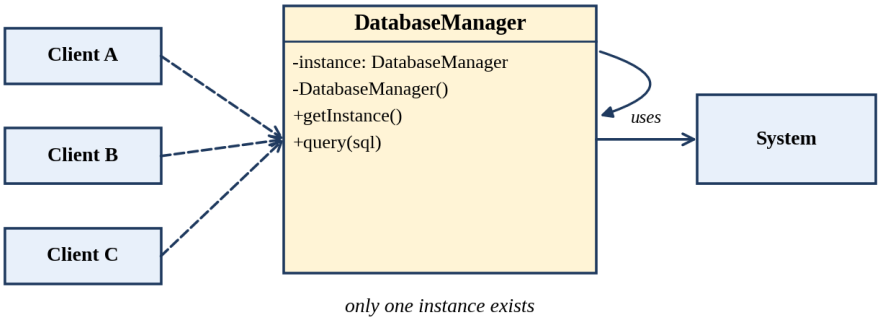


Figure 4: Singleton pattern: a single DatabaseManager instance shared by all clients

---

5.2 Factory Pattern

Purpose: Create objects dynamically without exposing object creation logic.

Possible Application

Examples:

Transportation System

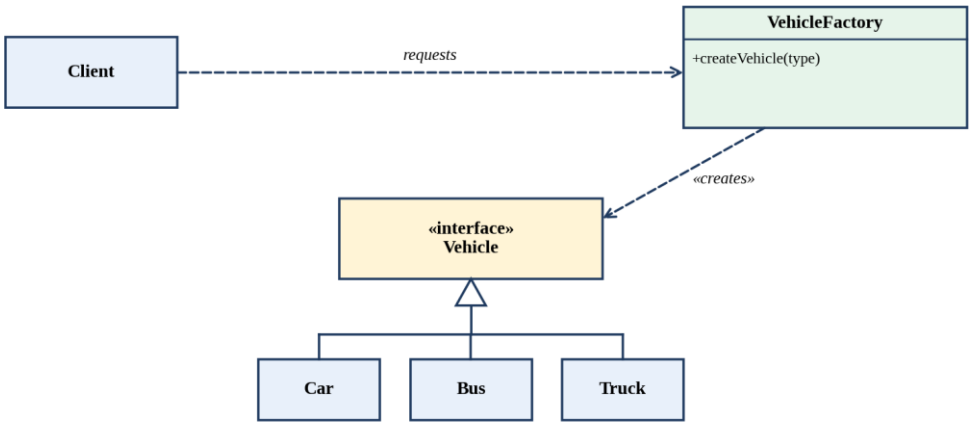


Figure 5: Factory pattern: VehicleFactory creating Car, Bus, and Truck objects

Payment System

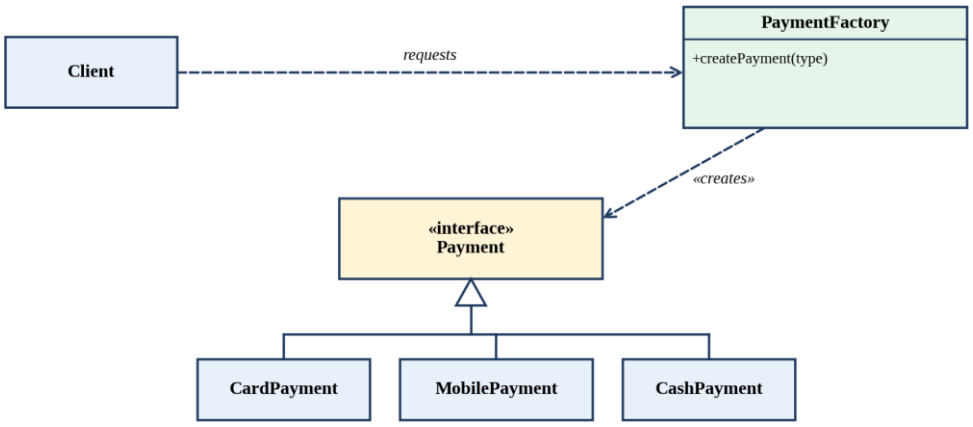


Figure 6: Factory pattern: PaymentFactory creating card, mobile, and cash payments

---

5.3 Observer Pattern

Purpose

Automatically notify dependent objects when a state changes.

Possible Applications

- Notification system
- Emergency alert
- Sensor monitoring

Example:

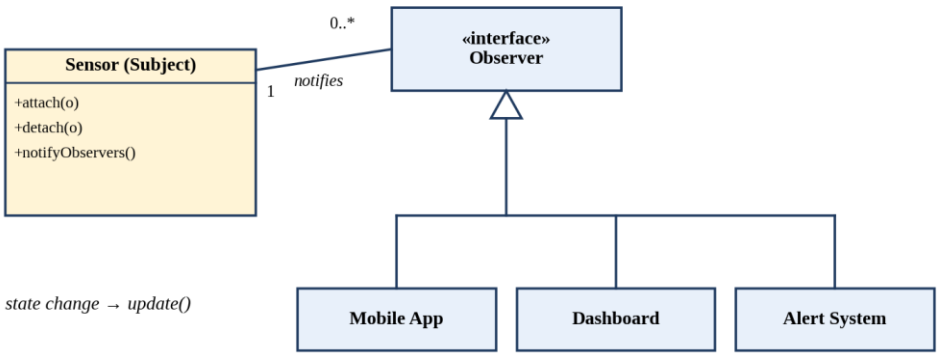


Figure 7: Observer pattern: Sensor notifying Mobile App, Dashboard, and Alert System

---

5.4 Strategy Pattern

Purpose

Allow different algorithms to be selected dynamically.

Possible Applications

Route Planning

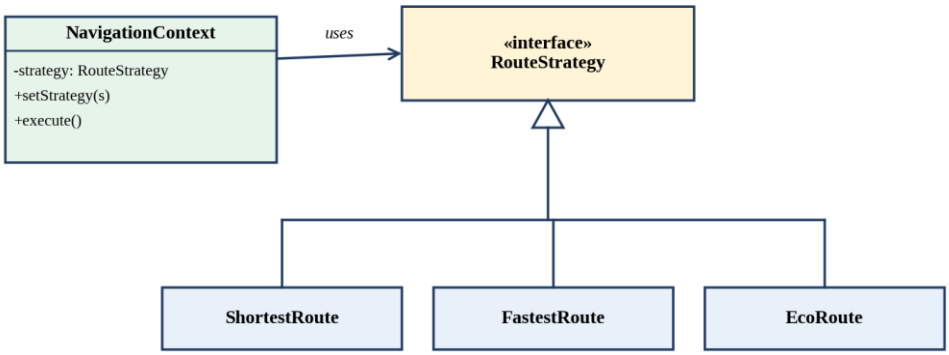


Figure 8: Strategy pattern: interchangeable route-planning algorithms

Agriculture

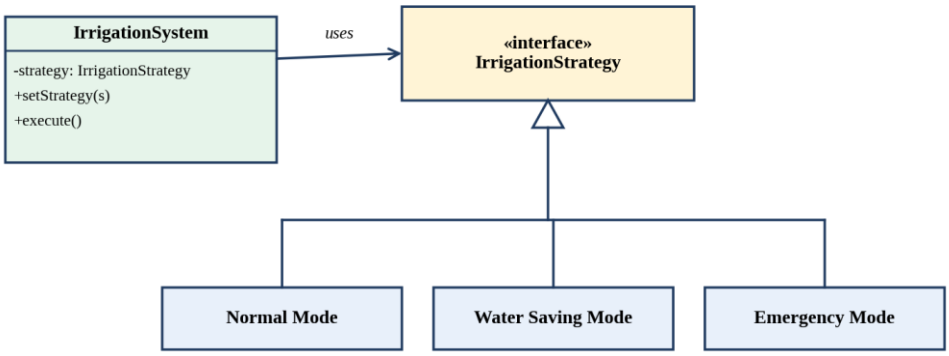


Figure 9: Strategy pattern: interchangeable irrigation modes

5.5 Adapter Pattern

Purpose

Allow incompatible systems to communicate.

Possible Applications

Weather System

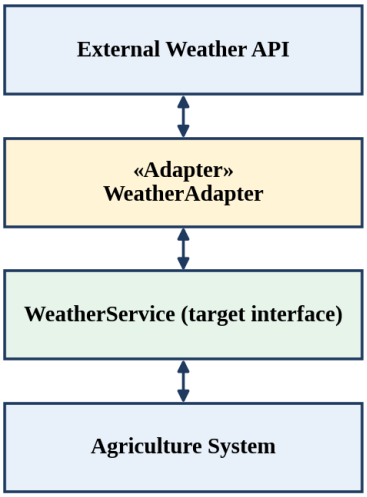


Figure 10: Adapter pattern: integrating an external weather API

Payment Gateway

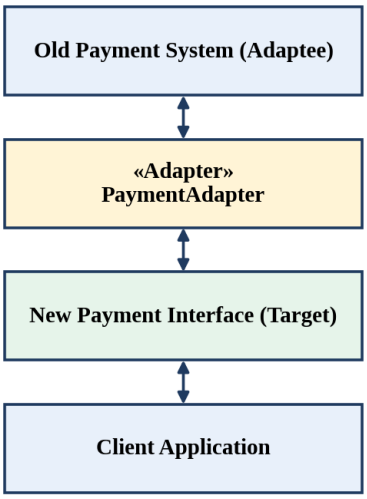


Figure 11: Adapter pattern: bridging an old payment system to a new interface

---

Final Lab Submission Format

Students must submit:

1. Project Background

2. System Objectives

3. System Actors

4. Object Identification

    4.1 Entity Objects

    4.2 Control Objects

    4.3 Boundary Objects

5. CRC Cards

6. UML Class Diagram

7. UML Sequence Diagram

8. Design Pattern Selection Explanation

9. Java Implementation

10. Testing Results

11. Conclusion

Project Proposal List

Project 1: Agricultural Automation System (AAS)

Project Overview

A smart agriculture platform that integrates IoT sensors, automation technologies, and intelligent decision-support mechanisms to improve farming efficiency.

Main Objectives

- Monitor crop and soil conditions
- Automate irrigation management
- Manage fertilizer application
- Detect pests and diseases
- Provide weather-based farming recommendations
- Support smart farming decisions

Main Actors

- Farmer
- Farm Administrator
- IoT Sensor
- Weather Service
- Drone System
- AI Analytics System

Major Entity Objects

- Crop
- Field
- SoilSensor
- WeatherData
- IrrigationPump
- FertilizerTank
- PestInformation
- Drone
- HarvestRecord

Possible Control Objects

- IrrigationController
- FertilizerScheduler
- PestDetectionManager
- HarvestPlanner
- FarmAnalyticsManager

Possible Boundary Objects

- FarmerMobileApp
- IoTSensorGateway

- WeatherAPIInterface
- DroneCameraInterface
- AlertNotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Farm Data Manager
Factory	Sensor creation
Observer	Weather/Pest alerts
Strategy	Irrigation strategy
Adapter	Weather API integration

Project 2: Dhaka Urban Management System (DUMS)

Project Overview

A smart city management platform designed to integrate urban services, citizen communication, IoT monitoring, and intelligent decision support.

Main Objectives

- Manage traffic congestion
- Improve waste collection
- Handle citizen complaints
- Support emergency response
- Monitor environmental conditions
- Generate urban analytics reports

Main Actors

- Citizen
- City Administrator
- Traffic Officer
- Waste Manager
- Emergency Officer
- IoT Sensor
- Government Authority

Major Entity Objects

- Citizen

- Complaint
- TrafficSignal
- Vehicle
- WasteBin
- WasteTruck
- EmergencyRequest
- Sensor
- Location
- Report

Possible Control Objects

- TrafficController
- WasteManagementController
- EmergencyResponseManager
- ComplaintManager
- UrbanAnalyticsEngine

Possible Boundary Objects

- CitizenMobileApp
- GovernmentDashboard
- TrafficCameraInterface
- GPSInterface
- NotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Central Urban Database
Factory	Urban Service Creation
Observer	Complaint Status Notification
Strategy	Route Optimization
Adapter	External Traffic API

Project 3: Smart Campus Bus Transportation System (SBTS)

Project Overview

A smart transportation platform that manages bus operations, passenger services, route optimization, and real-time monitoring.

Main Objectives

- Manage bus schedules

- Track buses using GPS
- Optimize routes
- Manage passenger tickets
- Provide real-time arrival information
- Monitor driver and vehicle performance

Main Actors

- Passenger
- Bus Driver
- Transport Manager
- Traffic Control Center
- GPS System
- Payment Service

Major Entity Objects

- Bus
- Driver
- Passenger
- Route
- BusStop
- Ticket
- Payment
- GPSLocation
- Schedule
- TrafficData

Possible Control Objects

- RouteManager
- ScheduleController
- TicketManager
- FareCalculator
- TrafficPredictionManager

Possible Boundary Objects

- PassengerMobileApp
- GPSInterface
- PaymentGateway
- TransportDashboard
- SMSNotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Transport Control Center
Factory	Ticket Type Creation
Observer	Bus Arrival Notification
Strategy	Route Selection Algorithm
Adapter	Payment Gateway Integration

Project 4: Smart University Campus Management System (SUCMS)

Project Overview

An integrated campus management system that manages students, courses, classrooms, transportation, security, and digital services.

Main Objectives

- Manage student information
- Handle course registration
- Manage classrooms and schedules
- Monitor campus security
- Provide smart notifications
- Manage digital campus services

Main Actors

- Student
- Faculty Member
- Administrator
- Security Officer
- Librarian
- IoT System

Major Entity Objects

- Student
- Faculty
- Course
- Classroom
- Schedule
- LibraryBook
- Attendance
- CampusCard
- SecurityCamera

- Notice

Possible Control Objects

- CourseRegistrationManager
- AttendanceController
- SecurityMonitoringSystem
- LibraryManager
- NotificationManager

Possible Boundary Objects

- StudentPortal
- FacultyPortal
- RFIDInterface
- LibrarySystemInterface
- MobileApplication

Recommended Design Patterns

Pattern	Application
Singleton	University Configuration Manager
Factory	User Account Creation
Observer	Notice/Result Notification
Strategy	Student Evaluation Strategy
Adapter	Library External System

Project 5: Smart Campus Food Management System (SFMS)

Project Overview

A smart food management platform that manages restaurants, food ordering, delivery, inventory, and customer services.

Main Objectives

- Manage food orders
- Track inventory
- Manage restaurant operations
- Optimize delivery routes
- Provide customer notifications
- Manage payment services

Main Actors

- Customer
- Restaurant Manager

- Delivery Agent
- Food Supplier
- Payment Service
- Administrator

Major Entity Objects

- Customer
- Restaurant
- FoodItem
- Order
- DeliveryAgent
- Inventory
- Payment
- Review
- Location
- Supplier

Possible Control Objects

- OrderManager
- InventoryController
- DeliveryManager
- RecommendationEngine
- PaymentController

Possible Boundary Objects

- CustomerMobileApp
- RestaurantDashboard
- PaymentGateway
- GPSInterface
- NotificationService

Recommended Design Patterns

Pattern	Application
Singleton	Order Management System
Factory	Payment Method Creation
Observer	Order Status Notification
Strategy	Delivery Route Selection
Adapter	Third-party Payment API

Final Assignment Distribution

Group	Assigned Project
Group 1	Agricultural Automation System
Group 2	Dhaka Urban Management System
Group 3	Smart Bus Transportation System
Group 4	Smart University Campus Management System
Group 5	Smart Food Management System

These five proposals have similar complexity levels and allow students to practice the complete OOAD workflow while applying the five required design patterns.

Lab Title: Design and Implementation of Object-Oriented Design Patterns for Real-World Software Systems

Lab Objective

After completing this laboratory, students will be able to:

1. Analyze real-world problems and identify software objects.
2. Develop CRC (Class Responsibility Collaborator) cards.
3. Design UML class diagrams and interaction diagrams.
4. Select appropriate object-oriented design patterns.
5. Implement design patterns using Java.
6. Understand how design patterns improve software flexibility, reusability, and maintainability.

Lab Workflow

Students must complete the following stages for the assigned project:

Phase 1: Requirement Analysis

Students analyze the given system scenario and identify:

- System objectives
- Main actors
- Functional requirements
- Major object entities

Phase 2: Object Identification

Students identify:

Entity Objects

Objects representing real-world concepts.

Example:

Student

Book

Vehicle

Patient

Product

Control Objects

Objects responsible for business logic and coordination.

Example:

PaymentManager

BookingController

NotificationManager

Boundary Objects

Objects interacting with external users or systems.

Example:

MobileApp

WebPortal

PaymentGateway

SensorInterface

Phase 3: CRC Card Development

Students prepare minimum:

- 6 CRC cards

Each CRC card must contain:

Class Name	Responsibilities	Collaborators
Student	Register course, view result	Course, Payment
Course	Store course information	Student, Faculty

Phase 4: UML Modeling

Students prepare:

1. UML Class Diagram

Must include:

- Classes
- Attributes
- Methods
- Relationships

Relationships:

- Association
- Aggregation
- Composition
- Generalization

2. UML Sequence Diagram

Show at least one important workflow.

Example:

User

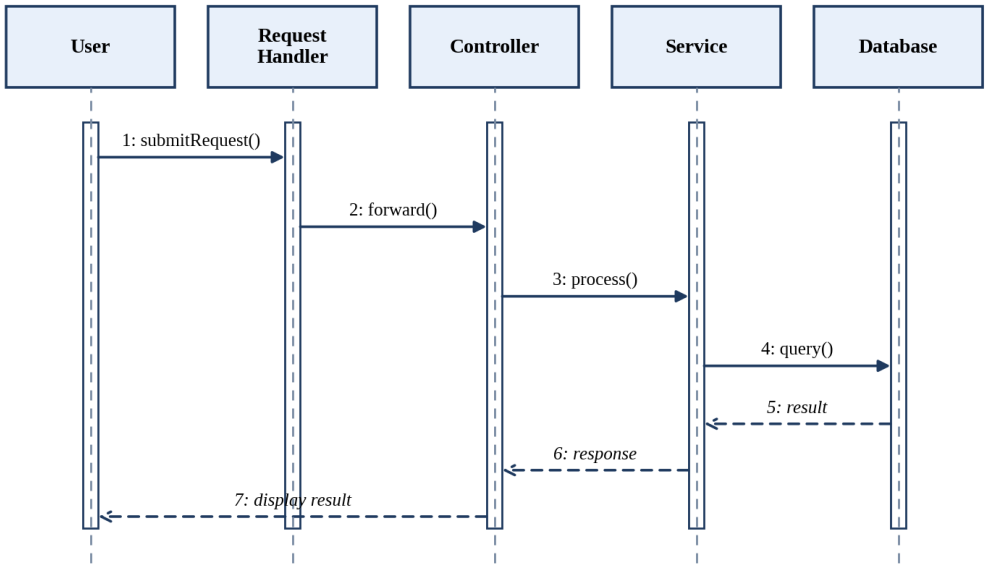


Figure 12: Sample UML sequence diagram: User request workflow

---

Phase 5: Design Pattern Implementation

Students must select and implement five design patterns from the following list:

Required Design Patterns

1. Singleton Pattern

Purpose:

Ensure only one instance of an object exists.

Possible use:

- Database connection
- Configuration manager
- System logger

Example:

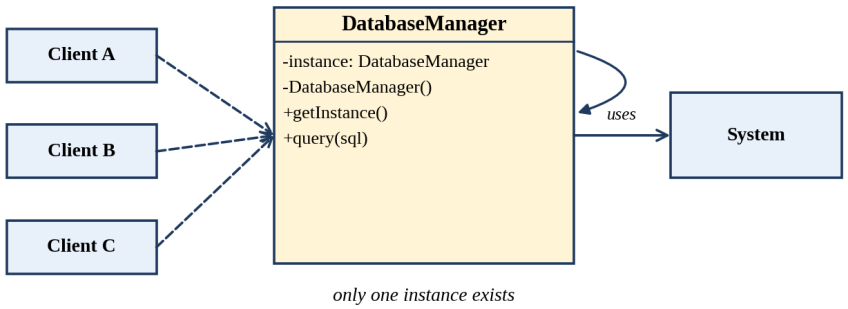


Figure 13: Singleton pattern: DatabaseManager

---

2. Factory Pattern

Purpose:

Create objects without exposing object creation logic.

Possible use:

- Vehicle creation
- Payment method creation
- User type creation

Example:

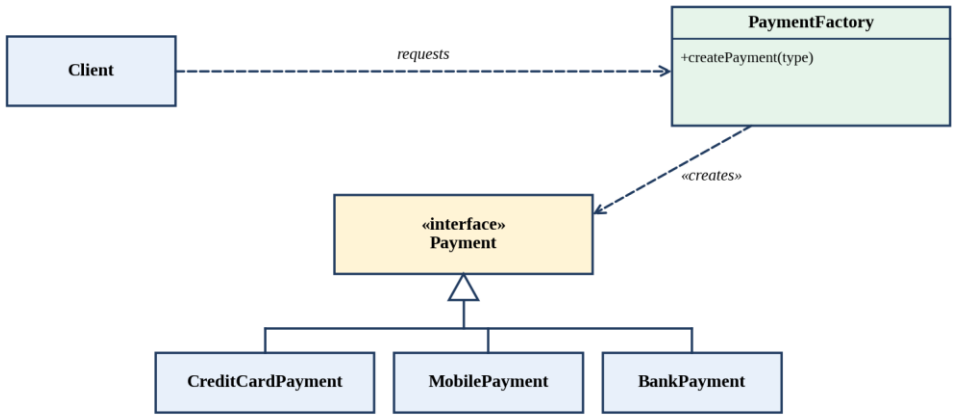


Figure 14: Factory pattern: PaymentFactory

---

3. Observer Pattern

Purpose:

Automatically notify dependent objects when a state changes.

Possible use:

- Notification system
- Alert system
- Stock monitoring

Example:

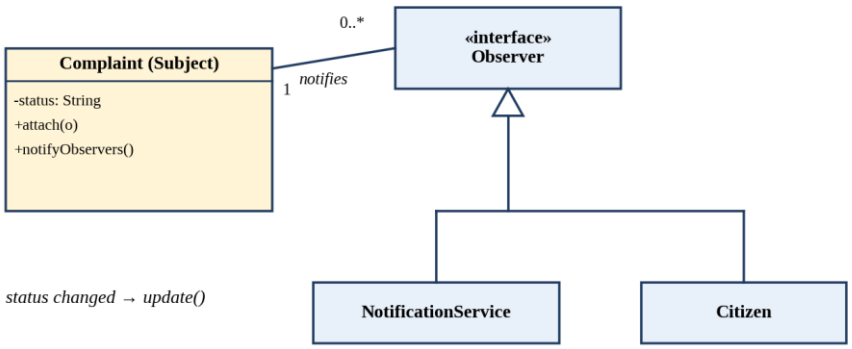


Figure 15: Observer pattern: complaint status notification

---

4. Strategy Pattern

Purpose:

Allow interchangeable algorithms.

Possible use:

- Route planning
- Sorting method
- Payment calculation
- Recommendation system

Example:

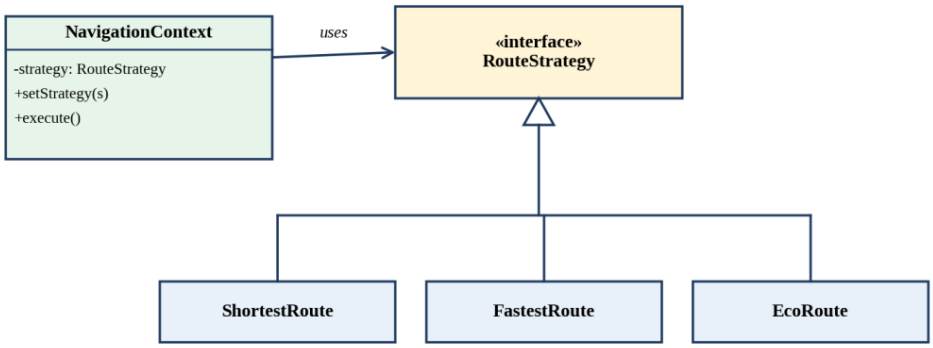


Figure 16: Strategy pattern: RouteStrategy

---

5. Adapter Pattern

Purpose:

Allow incompatible interfaces to work together.

Possible use:

- Third-party payment system
- External API integration
- Sensor connection

Example:

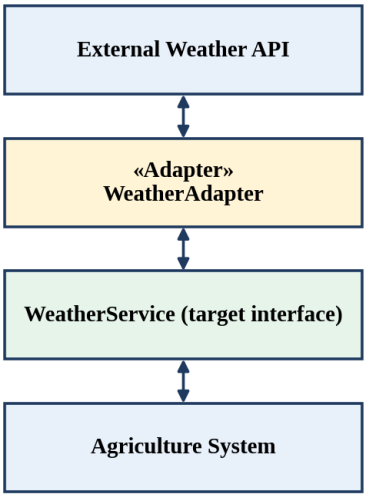


Figure 17: Adapter pattern: Weather API integration

Why avoid MVC?

You can mention in the manual:

MVC pattern is not included as a mandatory implementation pattern because it focuses mainly on software architecture separation and frequently requires database and user-interface development. This laboratory emphasizes object-oriented design patterns that demonstrate object creation, behavior management, communication, and flexibility.

Suggested Five Lab Projects

Project 1: Smart Hospital Management System (SHMS)

Background

A digital hospital platform manages patients, doctors, appointments, emergency services, and medical notifications.

Main Entities

Patient

Doctor

Appointment

MedicalRecord

Prescription

EmergencyRequest

Payment

Notification

Suggested Patterns

Pattern	Application
Singleton	Hospital database/configuration
Factory	User creation (Doctor, Patient, Staff)
Observer	Appointment notification
Strategy	Emergency priority calculation
Adapter	External laboratory system

Project 2: Online Food Delivery System

Entities

Customer

Restaurant

FoodItem

Order

DeliveryAgent

Payment

Location

Review

Patterns

Pattern	Application
Singleton	Order manager
Factory	Payment method
Observer	Order tracking notification
Strategy	Delivery route selection
Adapter	Online payment gateway

Project 3: Smart Agriculture Automation System

Entities

Crop

SoilSensor

WeatherData

IrrigationPump

Farmer

Fertilizer

Drone

Patterns

Pattern	Application
Singleton	Sensor data manager
Factory	Sensor creation
Observer	Weather alert
Strategy	Irrigation algorithm
Adapter	Weather API

Project 4: University Learning Management System (ULMS)

Entities

Student

Teacher

Course

Assignment

Exam

Result

Notification

Patterns

Pattern	Application
Singleton	Academic configuration
Factory	User account creation
Observer	Result publication notification
Strategy	Grading method
Adapter	Online exam integration

Project 5: Smart Transportation Management System

Entities

Vehicle

Driver

Route

TrafficSensor

Ticket

Payment

TrafficSignal

Patterns

Pattern	Application
Singleton	Traffic control center
Factory	Vehicle creation
Observer	Traffic alert
Strategy	Route optimization
Adapter	GPS service integration

Lab Examination Structure

Part A: Design (40 Marks)

Task	Marks
Requirement analysis	5
Object identification	10
CRC cards	10
UML class diagram	10
Sequence diagram	5

Part B: Implementation (50 Marks)

Task	Marks
Singleton implementation	10
Factory implementation	10
Observer implementation	10
Strategy implementation	10
Adapter implementation	10

Part C: Viva (10 Marks)

Sample questions:

1. Why did you select this object as an entity object?
2. Difference between aggregation and composition?
3. Why is Singleton suitable here?
4. What problem does Factory solve?
5. How does Observer reduce coupling?
6. How can Strategy improve system flexibility?

Recommended Lab Deliverable Format

Students submit:

1. Project Overview

2. Requirement Analysis

3. Actors

4. Entity-Control-Boundary Objects

5. CRC Cards

6. UML Class Diagram

7. UML Sequence Diagram

8. Design Pattern Selection Explanation

9. Java Implementation

10. Testing Results

11. Conclusion